MCP: from basics to production to enterprise

How the protocol works, what a production server needs, and how large companies serve hundreds or thousands of tools to AI agents.

1 · BASICS

The three roles, the building blocks, connection and discovery, one tool call, and what each side sees.

2 · PRODUCTION

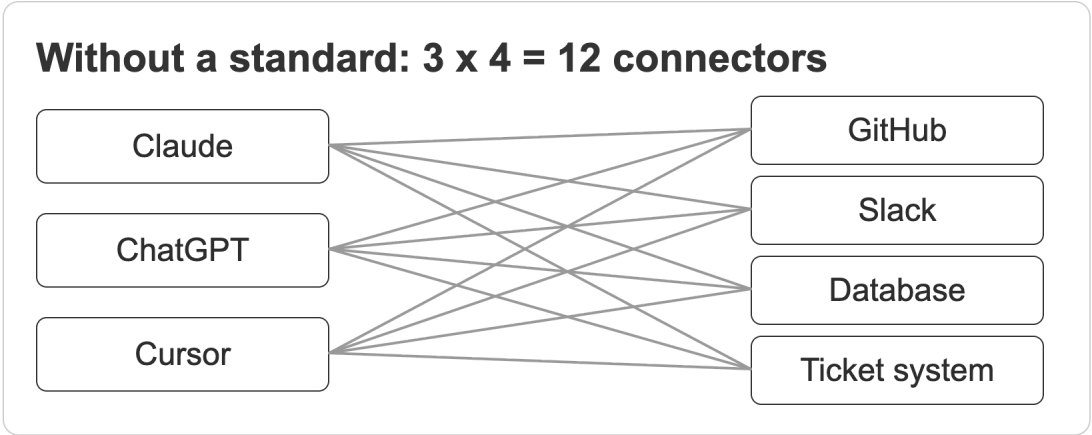
Sign-in, permissions, security, tool design, errors, files, sandbox, and logs.

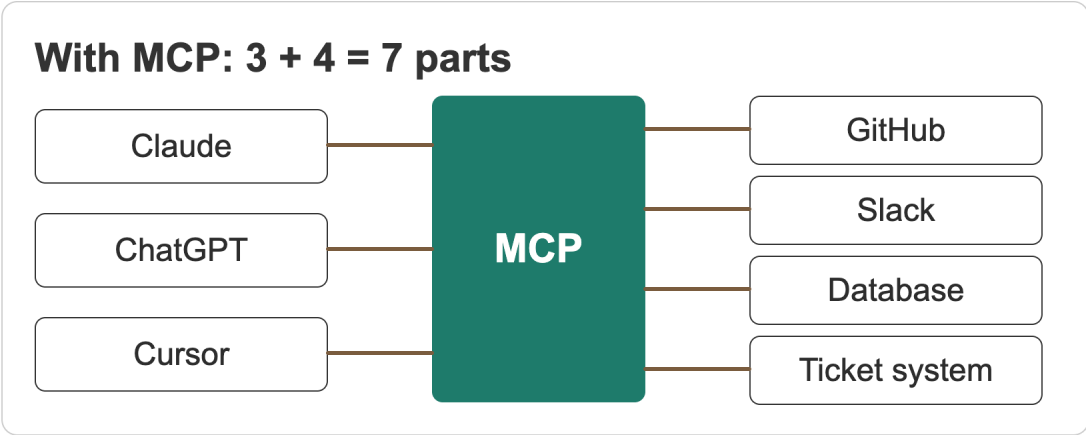
3 · SCALE AND ENTERPRISE

The problems with hundreds or thousands of tools, the best practices, and the design patterns.

Based on MCP specification revision 2026-07-28.

1.1 What is MCP





What MCP is

MCP (Model Context Protocol) is an open standard. It defines how an AI application finds tools and data on a server and how it calls them. It is not an agent framework and not a model API (application programming interface). Anthropic created it in November 2024. The Agentic AI Foundation has owned it since December 2025.

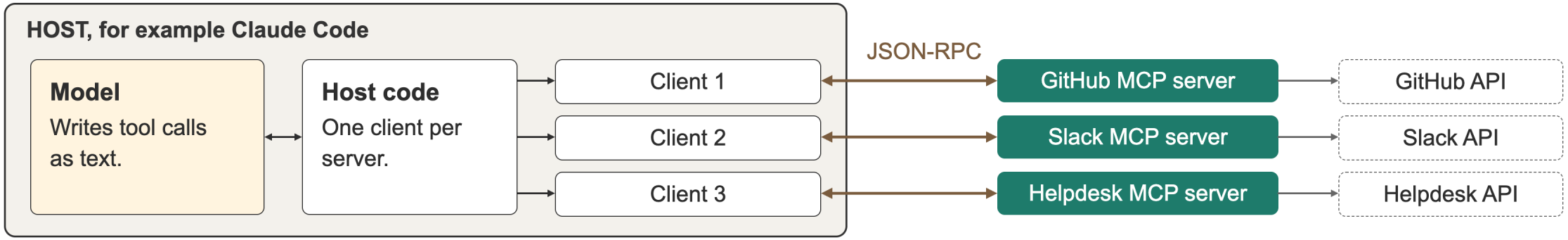
What JSON-RPC means

JSON-RPC 2.0 is the message format of MCP. RPC means remote procedure call: one program asks another to run a function. A request carries a method name, parameters, and an id. The response repeats the id and holds a result or an error.

```
Request:  {"jsonrpc": "2.0", "id": 1, "method": "tools/list"}
Response: {"jsonrpc": "2.0", "id": 1, "result": {"tools": [...]}}
Error:    {"jsonrpc": "2.0", "id": 1, "error": {"code": -32601}}
```

Sources: MCP specification 2026-07-28 · JSON-RPC 2.0 specification · MCP joins the Agentic AI Foundation

1.2 Host, client, and server



Host

The host is the application that the user opens. It holds the model, the conversation, and the permission rules. It reads the configuration and decides which servers to connect.

Examples: Claude Desktop, Claude Code, Cursor, VS Code, or an agent that your team builds.

Client

A client is a small piece of code inside the host. The host creates one client for each server. Each client keeps one connection, sends the requests, and returns the answers to the host.

Example: a host with 3 configured servers runs 3 clients at the same time.

Server

A server is a program that offers tools, resources, and prompts for one system. It receives requests, runs your code against your API or database, and returns results.

Examples: a GitHub server, a Slack server, and the Helpdesk server in this deck.

Why one client per server: each connection has its own credentials, its own protocol version, and its own failures. If the Slack server is down, the GitHub client still works. A server never sees the conversation or the traffic of another server.

Sources: MCP documentation: architecture · Claude Code MCP documentation

1.3 Tools, resources, and prompts

Tools

A tool is a function on the server that the model can ask to run. It has a name, a description, and an input schema (the list of allowed arguments).

EXAMPLE

search_tickets(status, priority) returns the matching tickets.

WHO SENDS WHAT

The host loads the tool list with tools/list. The model chooses a tool. The host sends tools/call.

Resources

A resource is a piece of data that the server can return, such as a file or a record. Its address is a URI (uniform resource identifier).

EXAMPLE

helpdesk://tickets/8841 returns the full text of ticket 8841.

WHO SENDS WHAT

The host loads the list with resources/list. The user or the host picks one. The host sends resources/read.

Prompts

A prompt is a ready-made message template that the server offers. The user picks it, often as a slash command in the chat.

EXAMPLE

/summarize_ticket 8841 returns a complete instruction for the model.

WHO SENDS WHAT

The host loads the list with prompts/list. The user picks one. The host sends prompts/get.

The main difference is who decides. The model decides when to call a tool. The host application decides when to read a resource. The user decides when to use a prompt. Most hosts support tools well. Support for resources and prompts differs from host to host, so a server must work with tools alone.

Sources: MCP specification: tools · MCP specification: resources · MCP documentation: architecture

1.4 How a host connects to a server

```
.mcp.json in the project folder (Claude Code)
{
  "mcpServers": {
    "helpdesk": {
      "type": "http",
      "url": "https://mcp.helpdesk.example/mcp",
      "headers": {"Authorization": "Bearer ${HELPDESK_TOKEN}"}
    },
    "files": {
      "command": "npx",
      "args": ["-y",
        "@modelcontextprotocol/server-filesystem", "."]
    }
  }
}
```

Same entry by command: **claude mcp add --transport http helpdesk <url>**

What each field means

`mcpServers`	The list of servers. Each key, such as helpdesk, is a name that you choose.
`type`	How messages travel. http means a remote server. No type means a local server.
`url`	The one address of the remote server. Every message is an HTTP POST to it.
`headers`	Added to every request. Here: the access token, read from an environment variable.
`command`	The program that the host starts on your machine for a local server.
`args`	Arguments for that program: -y installs without asking, then the package, then the folder it may read.

What happens next: the host creates one client per entry. Each client connects, signs in if the server requires it, and sends tools/list, resources/list, and prompts/list. Host code does this. The model and the user do nothing.

Sources: Claude Code MCP documentation · MCP specification: Streamable HTTP

1.5 What happens after you add a server

1 Save the configuration · YOU

Write the server entry into .mcp.json, or run claude mcp add. Nothing is sent to any server yet.

2 Start Claude Code and approve · YOU

For a server from the project file .mcp.json, Claude Code asks you once to approve it.

3 Connect in the background · CLAUDE CODE

Claude Code creates one client per server. A remote server without a valid token answers 401. Its status becomes: needs authentication.

4 Sign in with /mcp · YOU

Type /mcp, choose the server, and sign in in the browser. Claude Code stores the token for later sessions.

5 Discover · CLIENT TO SERVER

Each client asks its server what it offers: tools/list, prompts/list, and resources/list. No model is involved.

6 Use the tools · MODEL

At the start the model sees only the tool names. It loads a full definition when it needs one. Then it can ask for tools/call.

Two different things called /mcp

/mcp AT THE END OF THE URL

https://mcp.helpdesk.example/mcp is the address of the server. Its owner chose the path /mcp. Every request goes to this one path. tools/list is not a path: it is the method name inside the JSON body.

```
POST /mcp  always the same path
Mcp-Method: tools/list
{"jsonrpc": "2.0", "method": "tools/list", ...}
```

/mcp IN CLAUDE CODE

A command that you type in the chat. It lists each server with its status, such as connected, failed, or needs authentication. From there you sign in, reconnect, or turn a server off.

Sources: Claude Code MCP documentation · MCP specification: Streamable HTTP

1.6 How discovery works

The model never sends MCP requests. The client code inside the host sends them. Three requests are sent automatically. Three are sent only after the model or the user decides.

`tools/list`	**Automatic.** The client sends it after connecting. It returns every tool definition. The client sends it again when the list changes.
`tools/call`	**After the model decides.** The model chooses a tool and its arguments. The host checks permission. Then the client sends it.
`resources/list`	**Automatic.** It returns the list of resources, so that the host can show them in a menu.
`resources/read`	**After the user or the host decides.** It returns the content of one resource. The host adds it to the conversation.
`prompts/list`	**Automatic.** It returns the prompt templates, so that the host can show them as slash commands.
`prompts/get`	**After the user decides.** The user picks a prompt. It returns the finished messages for the model.

The same steps in Python, with the official SDK

```
from mcp import Client

url = "https://mcp.helpdesk.example/mcp"
async with Client(url) as client:
    # sent by host code, no model
    tools = await client.list_tools()
    # sent after the model chose a tool
    result = await client.call_tool(
        "search_tickets",
        {"status": "open"})
```

These methods are not separate URLs. A remote server has one endpoint, for example /mcp. The method name is a field in the JSON body of each request.

Sources: MCP specification: tools · MCP specification: resources · Claude Code MCP documentation · MCP Python SDK 2.2.0

1.7 How one tool call travels

1. **Discover**
   The client sends tools/list after connecting. The server returns all its tools.
2. **Translate**
   The host copies the definitions into the model prompt.
3. **Decide**
   The user asks. The model answers with a tool name and JSON arguments.
4. **Call**
   The host checks permission. The client sends tools/call to the server.
5. **Return**
   The server runs its code. The host gives the result text to the model.

_meta is a metadata field: data about the request, not arguments for the tool. It carries the protocol version, the client name, and the client's features.

```
Step 1. The tools/list result holds every tool of the server
{"tools": [{"name": "search_tickets", "description": "Search ...",
   "inputSchema": {...}}, ... all other tools follow here ...],
 "ttlMs": 0, "cacheScope": "private"}
Step 4. The tools/call request, recorded from the Python SDK
{"jsonrpc": "2.0", "id": 3, "method": "tools/call", "params": {
  "name": "search_tickets",
  "arguments": {"status": "open", "priority": "urgent"},
  "_meta": {
    "io.modelcontextprotocol/protocolVersion": "2026-07-28",
    "io.modelcontextprotocol/clientInfo":
        {"name": "mcp", "version": "0.1.0"},
    "io.modelcontextprotocol/clientCapabilities": {}}}}
Step 5. The result from the server
{"jsonrpc": "2.0", "id": 3, "result": {"resultType": "complete",
  "content": [{"type": "text", "text": "2 tickets: 8841, 8850"}],
  "isError": false}}
```

Sources: MCP specification: versioning · MCP specification: tools · Messages recorded from MCP Python SDK 2.2.0 on 11 October 2026, shortened

1.8 What a handshake is ☆ Optional deep dive

DEFINITION A handshake is a short exchange of messages at the start of a conversation between two programs. Both sides agree on the rules before the real data flows: who they are, which version they speak, and how they protect the data.

One request to an HTTPS server passes through these layers. Each layer has its own handshake, or none.

TCP the connection	`SYN → SYN-ACK → ACK` `3 messages`	TCP (Transmission Control Protocol) opens the connection. The client asks, the server agrees, the client confirms. This happens once per connection, and one connection can carry many requests.
TLS the encryption	`ClientHello → ServerHello, certificate, keys → ready`	TLS (Transport Layer Security) is the S in HTTPS. Both sides agree on the version and the encryption method. The server proves who it is with a certificate. TLS 1.3 needs one round trip.
HTTP the request	`request → response` `no handshake`	HTTP itself has no handshake. Each request carries everything: method, path, headers, and token. The server does not remember earlier requests.
MCP the application	`old: initialize → result` `new: none, _meta per request`	Until revision 2025-11-25, MCP added its own handshake on top: initialize, an answer, then initialized, and a session id. Revision 2026-07-28 removed it.

WHY MCP REMOVED IT A handshake creates a session that the server must remember. With 20 server instances behind a load balancer, every later request must reach the same instance. Without a handshake, any instance can answer any request.

Sources: TCP, RFC 9293 · TLS 1.3, RFC 8446 · MCP 2025-11-25: lifecycle · MCP changelog 2026-07-28

1.9 Who sees what

The model sees

Only what the host puts into the prompt: tool names, descriptions, input schemas, and later the result text. It never sees JSON-RPC, URLs, or tokens.

```
Prompt: {"name": "mcp__helpdesk__search_tickets", "description": "Search ...",
         "input_schema": {"type": "object", "properties": {...}}}
Model:  {"type": "tool_use", "name": "mcp__helpdesk__search_tickets",
         "input": {"status": "open", "priority": "urgent"}}
Result: {"type": "tool_result", "content": "2 tickets: 8841, 8850"}
```

The host and its clients see

Everything on the user's side: the configuration, the conversation, every tool list, every call and result, and the user's approvals. The host decides what enters the prompt.

```
Config:       helpdesk = https://mcp.helpdesk.example/mcp, files = npx ...
Tool lists:   6 tools from helpdesk, plus the tools from files
Conversation: "Which urgent tickets are open?"
Permissions:  search_tickets is allowed, create_refund must ask the user
```

The server sees

One HTTP request at a time: the method, the tool name, the arguments, and the access token. It does not see the conversation, the user's question, or other servers.

```
POST /mcp HTTP/1.1
Authorization: Bearer eyJhbGciOiJSUzI1NiIs...
MCP-Protocol-Version: 2026-07-28
Mcp-Method: tools/call
Mcp-Name: search_tickets
{"jsonrpc": "2.0", "id": 7, "method": "tools/call", "params": {"name": ...}}
```

Sources: Claude Code MCP documentation · MCP specification: Streamable HTTP · The model view uses the Claude API format, with ids left out

2.1 MCP in Production

1 Transport

The way messages travel between client and server. A remote server uses HTTP. A local server uses the input and output of a process.

2 Authentication and authorization

Authentication checks who the caller is. Authorization checks what this caller is allowed to do.

3 Tool design and validation

Tool design decides which tools the server offers and how the model sees them. Validation checks every argument before a tool runs.

4 Error handling

What the server returns when something fails. The model reads this text and uses it to decide its next step.

5 Reliability and human approval

Reliability keeps actions correct when calls are slow, repeated, or cut off. Human approval makes a person confirm a risky action.

6 Files and large results

How files go in and out, and how large results stay small enough for the model to read.

7 Sandbox

A closed environment around a program. It limits which files, network addresses, and secrets the program can reach.

8 Logging, tracing, and evaluation

Logging and tracing record what happened in every call. Evaluation tests if an agent can finish real tasks with the server.

Sources: MCP specification: tools · MCP security best practices · Claude docs: build an MCP server

2.2 The example: an online shop

Maya, a customer
She writes: cancel my order 5521.

→

Shopping assistant
The host, with the model and one MCP client.

→

Shop MCP server
https://mcp.shop.example/mcp

→

Shop database
Orders and customers.

Who	Order	Content	Status
Maya	5521	3 items, 120.00 USD	Paid, not shipped
Ravi	5530	1 item, 45.00 USD	Paid, not shipped

Tool of the server	What it does	Changes data
find_orders(status)	Lists the orders of the signed-in customer.	No
get_order(order_id)	Returns one order: status, items, and total.	No
cancel_order(order_id, reason)	Cancels an order that has not shipped. Starts the refund.	Yes

```
One request from start to end. The last line is a recorded answer.
Maya:       "Cancel my order 5521. I ordered it twice."
The model:  cancel_order(order_id="5521", reason="ordered twice")
The server: "Order 5521 is cancelled. Refund of 120.00 USD starts today."
```

The shop is invented for this deck. Its server was built and run with the MCP Python SDK 2.2.0 on 11 October 2026.

2.3 Authentication: who is calling

DEFINITION Authentication is the check of identity. The server finds out who sends a request before it runs any tool. The client proves its identity with a token: a secret string that stands for a signed-in user.

IN THE SHOP EXAMPLE

Maya tells her assistant to cancel order 5521. The assistant sends tools/call cancel_order to the shop server.

WITHOUT AUTHENTICATION

The URL is public. The server cannot tell Maya from a stranger. Any program can cancel any order. A 2026 study found no authentication on 40.55% of 7,973 public MCP servers.

WITH AUTHENTICATION

The assistant sends Maya's token with every request. The server checks the token first. Without a valid token the answer is 401, and no tool runs.

```
1. The request without a token
POST /mcp
Mcp-Method: tools/call, Mcp-Name: cancel_order
2. The recorded answer: 401, and where to sign in
401 Unauthorized
WWW-Authenticate: Bearer error="invalid_token",
  resource_metadata="https://mcp.shop.example/
  .well-known/oauth-protected-resource/mcp"
3. The same request with Maya's token
Authorization: Bearer tok_maya
200 OK "Order 5521 is cancelled. Refund of
  120.00 USD starts today."
```

Bearer token
Whoever carries the token is accepted. Keep it secret and send it only over HTTPS.

401
The HTTP status code for: no valid token. The client must sign in first.

OAuth
The standard sign-in flow. The client gets a token and never sees the password.

Sources: MCP specification: authorization · Study of 7,973 servers · Recorded on a local test server, host name replaced

2.4 Authentication: the techniques

Authorization server
Signs people or programs in. Issues the tokens.

token →

Client (the assistant)
Keeps the token. Sends it with every request.

request →

MCP server
Checks every token. Never issues one.

Technique	Who signs in, and how	Client side	Server side	Choose it when
1. API key	No sign-in service. The shop creates one fixed secret and gives it to the client.	Keep the key in an environment variable. Send it in a header.	Compare it with the stored key. Replace keys on a schedule.	Your own scripts and tools, with no user.
2. OAuth 2.1 sign-in	A person signs in. Maya logs in in her browser and approves the assistant. The token says: this is Maya.	Open the browser. Keep the token. Refresh it when it expires.	Check signature, issuer, audience, and expiry. Read the user.	A person uses the assistant, like Maya.
3. OAuth client credentials	A program signs in. A nightly job sends its own id and secret. The token says: this is the job.	Keep the id and the secret in a secret store.	The same checks. Give the program only the scopes it needs.	Jobs with no person. Draft MCP extension.
4. Company single sign-on	An employee is signed in at the company identity provider (IdP). The client exchanges that sign-in for a token.	Exchange the company sign-in for a server token.	Accept only tokens that the company IdP approved.	Employees, with access managed by the company.

The difference between 2 and 3: in 2 a person signs in in a browser. In 3 a program signs in with its own secret.

Sources: MCP specification: authorization · Client credentials · Enterprise sign-on · Claude Code docs

2.5 Authorization: what the caller may do

DEFINITION Authorization is the check of permission. The server already knows who the caller is. Now it decides if this caller may do this action on this object. The server decides in its own code, on every call.

THE THREE CHECKS, IN THIS ORDER

1 Scope

Does the token allow this kind of action? A scope is a named permission in the token, such as orders:write.

2 Owner

Does order 5521 belong to the caller? The server takes the user from the token, never from the tool arguments.

3 Rule

Is the action allowed for this order now? An order that has shipped cannot be cancelled.

Caller	Call	Recorded result
Maya	get_order(5521)	Order 5521: paid, 3 items, 120.00 USD
Maya	get_order(5530), the order of Ravi	NOT_FOUND: No order 5530 in your account.
Ravi (read only)	cancel_order(5530)	PERMISSION_DENIED: Your sign-in allows reading orders only.
Maya	cancel_order(5521)	Order 5521 is cancelled. Refund of 120.00 USD starts today.

WHY NOT_FOUND Maya must not learn that order 5530 exists, so the answer is the same as for an order that does not exist. A real case: Asana's MCP server showed data of one customer organization to another from 1 May to 4 June 2025. Asana called it a logic flaw.

Sources: MCP security best practices · Asana incident · Results recorded on a local test server

2.6 Authorization: the techniques

Five techniques. Every server needs the first two. All of them run in server code, because a model can be tricked and a client can be replaced.

Technique	How it works in the shop
1. Scope per tool	Each tool needs a scope, a named permission in the token. cancel_order needs orders:write. Ravi has only orders:read.
2. Owner check	Every query filters by the user from the token. Order 5530 does not exist for Maya.
3. Roles	A role is a named set of permissions. A customer cancels own orders. A support agent cancels any order.
4. Hidden tools	tools/list returns only the tools that the token can use. GitHub's server hides write tools from read-only tokens.
5. Step-up	A token lacks a scope. The server answers 403 with the missing scope. The client asks the user for it and tries again.

SERVER CODE OF THE SHOP EXAMPLE

```
@mcp.tool()
def cancel_order(order_id: str, reason: str) → str:
    user = get_access_token()  # from the token
    # technique 1: scope
    if "orders:write" not in user.scopes:
        raise ToolError("PERMISSION_DENIED: ...")
    # technique 2: owner
    order = ORDERS.get(order_id)
    if order is None or order["owner"] ≠ user.subject:
        raise ToolError("NOT_FOUND: ...")
    order["status"] = "cancelled"
    return f"Order {order_id} is cancelled."
```

This code produced the recorded results on the slide before.

Sources: MCP specification: authorization · MCP specification: tools · GitHub's server · Code run with MCP Python SDK 2.2.0

2.7 Tool design: build tasks, not endpoints

```
Endpoint style: 4 calls
list_users(name="Maya")
list_tickets(assignee_id=17)
get_ticket(id=8841)
get_ticket(id=8850)
```

The model must carry ids between calls. Each call adds a round trip and more tokens.

```
Task style: 1 call
search_tickets(assignee="Maya",
    priority="urgent", status="open")
```

One call answers the question: which urgent tickets wait for Maya?

Block rebuilt its Linear server from more than 30 tools to 2. The same question went from 4 to 6 calls down to 1.

Names

Use verb_noun with a service prefix, for example helpdesk_search_tickets. A host may load 20 servers, and names must stay unique.

Descriptions

Say what the tool does, when to use it, and when not. Put the main rule first: Claude Code cuts each description at 2,048 characters.

Input schema

Few parameters, with defaults and enums (fixed lists of allowed values). The model cannot guess the spelling of a free-text status.

Results

Return what the next step needs: names and ids, not 40 raw fields. GitHub removed more than 75% of the output tokens of list_pull_requests this way.

Annotations

readOnlyHint and destructiveHint help a host decide when to ask the user. They are hints from the server, not a security control.

Sources: Anthropic: writing tools for agents · Block's playbook · GitHub talk at AI Engineer

2.8 Error handling: errors the model can act on

DEFINITION Error handling decides what the server says when a call fails. A classic program reads a status code. An agent reads the error text: it goes into the model's context like any result, and the model chooses its next step from it.

IN THE SHOP EXAMPLE Two days later, Maya asks again: cancel order 5521. The order shipped yesterday.

A classic API error

```
409 Conflict
```

WHAT THE MODEL DOES NEXT

It only knows that something failed. It guesses: it sends the same call again, or it tells Maya that something went wrong.

An error written for the model

```
ORDER_ALREADY_SHIPPED: Order 5521 shipped
on 10 October, so it cannot be cancelled.
Nothing was changed. Use request_return
for this order instead.
```

WHAT THE MODEL DOES NEXT

It calls request_return, another tool of the shop server, and tells Maya that her return has started.

WHY IT MATTERS Anthropic advises errors that "communicate specific and actionable improvements, rather than opaque error codes or tracebacks". Claude retries an invalid call 2 to 3 times with corrections, so the error text decides if a retry succeeds.

Sources: Anthropic: writing tools for agents · Claude docs: handle tool calls · Alpic: better error responses

2.9 Error handling: the techniques

Protocol error: no tool ran

The request itself is broken: unknown tool name, broken JSON, or an unsupported protocol version. The answer is a JSON-RPC error object. The host handles it. The model usually does not see it.

Tool error: the tool ran but failed

Wrong argument, order not found, order shipped, service down. The answer is a normal result with isError: true and a text for the model. Since revision 2025-11-25, wrong arguments are tool errors too, so the model can fix them.

Case in the shop	The error text that the server returns	What the model does next
Wrong argument	INVALID_ARGUMENT: reason "too expensive" is not allowed. Use one of: ordered_twice, wrong_item, late, other. Nothing was changed.	Calls cancel_order again with reason "other".
Wrong tool for this state	ORDER_ALREADY_SHIPPED: Order 5521 shipped on 10 October. Nothing was changed. Use request_return instead.	Calls request_return for order 5521.
Temporary failure	REFUND_SERVICE_DOWN: The refund service did not answer. Nothing was changed. Try again in 30 seconds. After 3 failures, tell the customer.	Waits, tries once more, then tells Maya that the refund will follow.

Four parts in every error: what went wrong, whether anything changed, what to do next, and a short code word for logs. **Never include** stack traces, database queries, secrets, or data of other customers.

Sources: MCP specification: tools · SEP-1303 · Alpic: better error responses · request_return is an extra tool of the shop

2.10 Reliability: retries, long work, and human approval

Safe retries

A broken connection hides whether the refund happened, so the model calls again. Accept an idempotency_key: a repeat with the same key returns the first result and changes nothing. Stripe's API works this way.

Timeouts and long work

Hosts stop waiting: Claude apps allow 240 seconds per tool call. If work can run longer than about 30 seconds, start a job, return a job id at once, and offer a status tool.

Rate limits

An agent in a loop can send hundreds of calls in a minute. Limit per user, tenant, and tool. Say when to retry: RATE_LIMITED: try again in 30 seconds.

Human approval: the server enforces two steps

Host prompts are weak protection: Claude Code users approve 93% of permission prompts. For refunds and payouts, Stripe's server waits for a person.

STEP 1
The model calls create_refund for order 8841, 120 USD.

→

STEP 2
The server does not pay yet. It returns an approval URL.

→

STEP 3 · PERSON
A person opens the URL and approves. The approval expires after 24 hours.

→

STEP 4
The model calls again. The server pays once.

Sources: Stripe MCP docs · Stripe: idempotent requests · Anthropic: auto mode · Claude docs: limits

2.11 Files and large results

MCP has no file upload feature in any released revision. If a file travels as base64 (bytes written as text) in tool arguments, the model must write every byte: about 699,000 output tokens (estimate) for a 2 MB screenshot.

Upload with a pre-signed URL: the bytes never enter the model context

STEP 1
The model calls create_upload(name, size).

→

STEP 2
The server returns an upload URL (valid for 300 seconds, with a size limit) and a file_id.

→

STEP 3 · HOST
The host or the user sends the bytes to storage with HTTP PUT.

→

STEP 4
The model calls attach_file(ticket_id, file_id).

Files out

Return a short-lived download link or a resource URI, with a small preview. Do not return the bytes.

Pagination

Return 20 items, a total, and a cursor (a marker for the next page). Start with a summary: 37 tickets match, 5 urgent.

Host limits

Claude Code: 25,000 tokens per tool result by default. Claude apps: about 150,000 characters. Stay below both.

Working files

With code execution, tools write big data to a file and return the path. Anthropic: one workflow fell from 150,000 to 2,000 tokens.

Sources: Claude docs: limits · Claude Code MCP documentation · Anthropic: code execution with MCP

2.12 Sandbox: limit what code can reach

An agent that runs code or local servers can be tricked by injected text. A sandbox keeps the damage small even when the model is fooled.

File system wall
Read and write only inside the project folder. No home directory, no SSH keys.

Network wall
Outbound traffic goes through a proxy that allows only a list of known domains.

Credential wall
Secrets stay outside the sandbox. A proxy or gateway adds them to requests, so the code never holds them.

Approach	Real example
OS sandbox	Claude Code uses Linux bubblewrap and macOS seatbelt for file and network isolation. Anthropic measured 84% fewer permission prompts in internal use.
Containers	Docker MCP Gateway runs each server in its own container: 1 CPU, 2 GB of memory, no host files by default.
Isolates	Cloudflare runs model-written code in V8 isolates (light sandboxes inside the JavaScript engine). fetch() is blocked. Code can reach only the connected MCP tools.

A sandbox does not replace authorization. The server still checks every call.

Sources: Anthropic: Claude Code sandboxing · Cloudflare: Code Mode · Docker MCP Gateway

2.13 Logging, traces, and evaluation

One structured log line per tool call

```
{"time": "2026-10-10T07:41:03Z", "trace": "4bf92f35",
 "tenant": "acme", "user": "u_17",
 "client": "claude-code", "tool": "create_refund",
 "duration_ms": 412, "result_tokens": 96,
 "error": "PERMISSION_DENIED"}
```

Do not log secrets or full arguments with personal data. MCP's own logging feature is deprecated, so send logs to your normal log system.

Traces with OpenTelemetry

OpenTelemetry is an open standard for traces and metrics. Its MCP conventions name each span after the call, for example tools/call create_refund, with attributes such as mcp.method.name and gen_ai.tool.name. One trace joins the host, the server, and the upstream API.

Metrics to watch

- Calls and error rate for each tool and each error code
- Latency (95th percentile) for each tool
- Result size in tokens
- Recovery rate: a successful call within five minutes after a failure (Amplitude)
- Tools that agents never call

Evaluate with real tasks

Write realistic tasks that need several tool calls and have a known answer. Run an agent on them before every release. Measure task success, number of calls, and tokens. Read the transcripts: they show where a description or an error misleads the model.

Keep a separate audit log: who, which tenant, which tool, which object, and who approved.

Sources: OpenTelemetry: MCP conventions · Anthropic: writing tools for agents · Amplitude

3.1 The enterprise problem: too many tools

THE SETUP In a large company one agent connects to many servers: orders, billing, helpdesk, HR, code. The client sends every tool definition (name, description, input schema) to the model on every request, before the user types anything. One definition costs about 150 to 950 tokens (a token is about 4 characters of text).

HOW MUCH OF A 200,000-TOKEN CONTEXT WINDOW THE DEFINITIONS TAKE



No room for the work

Less space for the chat, files, and results. Results fill it too: a 2-hour meeting transcript copied from one tool to another adds about 50,000 tokens.

Wrong tool chosen

Anthropic: choice gets worse past 30 to 50 tools, and look-alikes such as orders_cancel and subscriptions_cancel get mixed up. Claude Opus 4.5, all tools loaded: 79.5% right. With tool search: 88.1%.

Paid on every request

Claude Opus 5.5 costs $4 per million input tokens. 55,000 tokens of definitions over a 50-step task: $11.00. Prompt caching cuts this to about $0.81, but the space is still used.

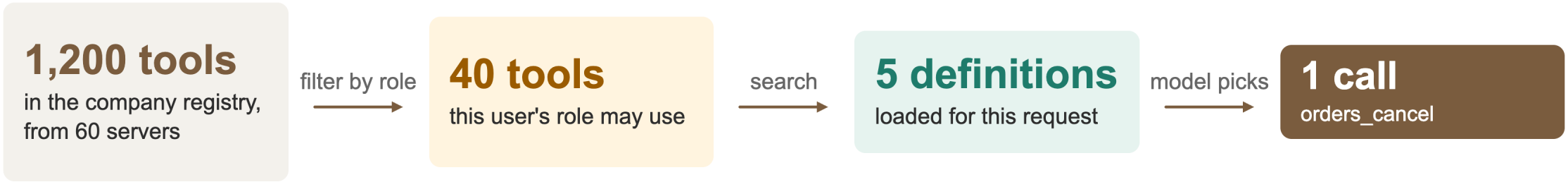
Limits and name clashes

VS Code allows at most 128 tools per request. Tool names are unique only inside one server, so two servers can both offer a tool called search.

Sources: Anthropic: advanced tool use · Tool search docs · S. Spence · Speakeasy · Anthropic: code execution · Claude prompt caching · VS Code

3.2 Best practices for hundreds or thousands of tools

THE GOAL: THE MODEL SEES ONLY WHAT THIS REQUEST NEEDS (an illustration)



1 Build task-shaped tools

Merge small API wrappers into a few tools that finish a task. Amplitude: 9 cohort tools became 1. Block: 30+ tools became 2.

2 Name and describe clearly

Prefix by service: orders_cancel, billing_refund. Say when to use a tool and when not to. Search and the model both read this text.

3 Show each user only their tools

Group tools into toolsets and filter by role. GitHub loads about 40 of its 100+ tools by default: about 49% less context.

4 Load definitions only when needed

Do not send every definition up front. Use tool search or a pattern from 3.3. Keep the 3 to 5 most used tools always loaded.

5 Keep results small

Return summaries and pages. Let code handle big data. Anthropic: one workflow fell from 150,000 to 2,000 tokens.

6 Measure with real requests

Test search and tool choice before each release. Keyword search over 4,027 tools found the right tool only 56% to 64% of the time.

Sources: Amplitude · Block · GitHub · Anthropic: advanced tool use · Anthropic: code execution · Arcade

3.3 Design patterns for a large tool catalog

Tool search — CLIENT

search "cancel order" ▸ 3 to 5 tools load ▸ model calls one

Result: Anthropic: 77,000 tokens fell to 8,700. On by default in Claude Code.

Cost: Search can miss a tool with a poor description.

Meta-tools — SERVER

search_tools ▸ describe_tools ▸ execute_tool

Result: Only these 3 tools are listed. Speakeasy, 400 tools: 405,100 tokens at the start fell to about 2,500.

Cost: 2 to 3 times more calls, about 50% more time.

Code mode — SERVER

model writes code ▸ sandbox runs it ▸ only the answer returns

Result: Cloudflare: 2,500+ API endpoints in about 1,000 tokens, not 1.17 million.

Cost: Needs a secure sandbox. Each action is harder to review.

Agent as a tool — SERVER

main agent ▸ one tool: a small agent ▸ short answer

Result: The small agent holds all the real tools. Sentry: about 14,000 tokens fell to about 720.

Cost: Slower: 11.29 s became 23.81 s per answer.

Gateway — BETWEEN

agents ▸ gateway ▸ servers

Result: One entry point does sign-in, role filter, search, and audit. Uber: 10,000+ services. Cloudflare: up to 80 servers per portal.

Cost: One more hop to run and secure.

Which pattern at which size

Up to 30 tools: load all, design them well.

30 to 100: toolsets and tool search.

100 to 500: search, meta-tools, or code mode behind a gateway.

500+: code mode, or search tested on your own requests.

Sources: Anthropic · Speakeasy · Cloudflare · Sentry · Uber (talk summary) · Cloudflare portals · Size bands: our reading